

JS Array Iteration Methods

1. What Are Array Iteration Methods?

- a. Array iteration methods in JavaScript are built-in functions that allow us to loop through arrays and perform specific operations on each element.
- b. These methods simplify array manipulation tasks such as transforming elements, filtering values, reducing arrays to a single value, or simply executing a function on every element.

2. Types of Array Iteration Methods:

- a. There are 3 main popular types of array iteration methods used for Array:
 - i. `map()`
 - ii. `filter()`
 - iii. `reduce()`
- b. We will see each and every method in detail. Let's see:

3. `map()`:

- a. The `map()` method creates a new array by applying a given function to every element in the array.
- b. It does *not modify* the *original array*.
- c. It iterates through each element of the array and invokes a callback function for each element.
- d. The result of the callback function is then added to the new array.

e. **Syntax:**



```
array.map((cValue, cIndex, cArray) => {  
    // return new value for the current element  
});
```

f. **Parameters:**

- i. **currentValue**: The current element being processed.
 - ii. **currentIndex** (optional): The index of the current element.
 - iii. **currentArray** (optional): The original array.
- g. **Return Type**: Returns a new array with transformed elements.

4. **filter()**:

- a. The **filter()** method creates a new array containing all the elements that pass a certain condition (provided in a callback function).
 - b. It iterates through each element of the array and invokes the callback function for each element.
 - c. If the callback function returns **true** for an element, that element is included in the new array; otherwise, it is excluded.
 - d. It filters out unwanted values.
- e. **Syntax:**



```
array.filter((cValue, cIndex, cArray) => {  
    // return true to keep the element, false otherwise  
});
```

f. **Parameters:**

- i. **currentValue**: The current element being processed.
 - ii. **currentIndex** (optional): The index of the current element.
 - iii. **currentArray** (optional): The original array.
- g. **Return Type**: Returns a **new array** containing elements that satisfy the condition.

5. **reduce()**:

- a. The **reduce()** method executes a reducer function on each element of the array, resulting in a single output value.
- b. The callback function takes **four arguments**: accumulator, currentValue, currentIndex, and the array itself.
- c. It is ideal used for aggregating array elements into a sum, product, or other computation.
- d. **Syntax**:



```
array.reduce((accumulator, cValue, cIndex, cArray) => {  
    // logic to aggregate values  
  
    return updatedAccumulator;  
},  
initialValue);
```

e. **Parameters:**

- i. **accumulator**: The accumulated result from the previous iterations.
- ii. **currentValue**: The current element being processed.
- iii. **currentIndex** (optional): The index of the current element.

- iv. **currentArray** (optional): The original array.
- v. **initialValue** (optional): The initial value of the accumulator (default is the first element of the array).
- f. **Return Type**: Returns the *final accumulated (Single) value*.

6. **forEach()**:

- a. The **forEach()** method executes a provided function once for each array element.
- b. Unlike **map()**, it does not return a new array.
- c. **Syntax**:



```
array.forEach((cValue, cIndex, cArray) => {  
    // perform an operation for each element  
});
```

d. **Parameters**:

- i. **currentValue**: The current element being processed.
 - ii. **currentIndex** (optional): The index of the current element.
 - iii. **currentArray** (optional): The original array.
- e. **Return Type**: **Undefined**. It does not return anything.

Comparison Table map(), filter(), reduce() and forEach():

Comparison of Array Methods			
Method	Modifies Original Array?	Returns New Array?	Use Case
map()	No	Yes	Transform elements into a new array.
filter()	No	Yes	Extract elements based on a condition.
reduce()	No	No (Single Value)	Aggregate elements into one value.
forEach()	Yes (if manually modified)	No	Perform side effects like logging or updates.